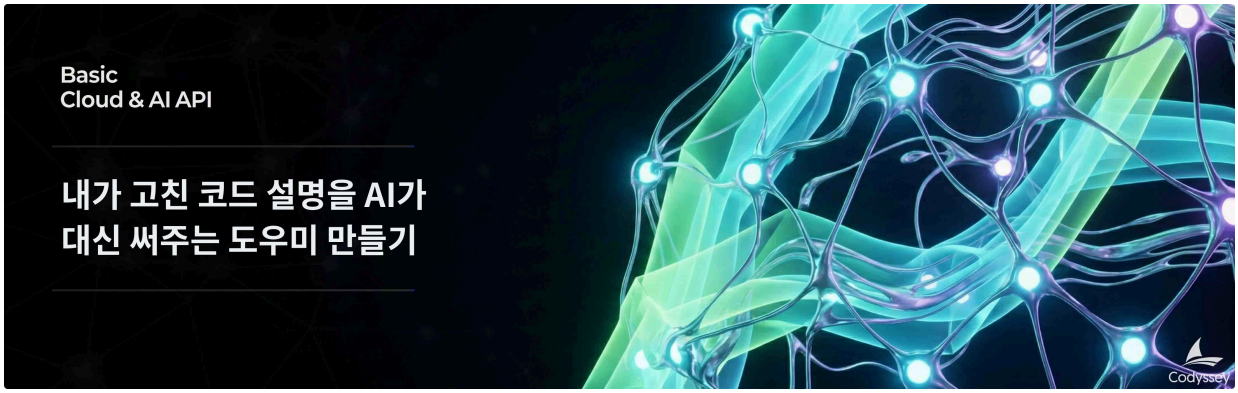
분야 AI/SW 기초 구분 클라우드와 AI API 학습시간 40시간 필수/선택 필수

내가 고친 코드 설명을 AI가 대신 써주는 도우미 만들기

문제기술

기술적 설명



1. 미션 소개

AI API 연동이라고 하면 거창하게 들리는데, 사실 핵심은 '어떤 입력을 주냐'에 있습니다. git diff를 넘기고, 의미 있는 커밋 메시지와 PR 설명이 나오게 프롬프트를 설계하는 것이 이번 미션입니다. 단순 API 호출이 아니라, AI 결과물의 품질을 직접 제어하는 방법을 경험합니다.

AI API는 외부 AI 모델의 텍스트 생성 기능을 애플리케이션에서 호출해 사용할 수 있게 해주는 방식입니다. 고객 응대 문장 생성, 문서 요약, 개발 로그 정리처럼 입력값을 받아 AI가 생성 결과를 반환하는 자동화 작업에 활용됩니다.

이번 미션에서는 AI API를 호출하는 Python 코드를 기반으로, Git 변경 사항을 입력값으로 받아 커밋 메시지와 PR 설명을 자동 생성하는 CLI 도구를 개발합니다. 단순 호출에 그치지 않고, API 파라미터와 컨텍스트(Commit/PR 양식, 변경 이유, 요구사항)를 설계해 원하는 품질의 결과가 나오도록 프롬프트를 최적화합니다.

또한 Git 명령 실행 결과를 프로그램에 연결하고, 생성된 결과물을 실제 커밋/PR 작성 흐름에 적용하며 연동·검증·자동화를 하나의 흐름으로 구현합니다. 이를 통해 AI 기능을 서비스에 붙이는 방식을 구현 관점에서 이해하고, 이후 다양한 업무 자동화/개발 보조 도구로 확장할 수 있는 기반을 갖추게 됩니다.

2. 최종 결과물

다음 3가지 결과물을 완성한다.

1. **AI API 연동 및 자동화 흐름 동작 확인**

- 입력/요청: AI API Key를 환경변수로 설정한 뒤, 프로젝트 루트 디렉토리에서 커밋/PR 자동 생성을 수행하는 Python CLI 프로그램을 실행한다.
- 출력/화면: Python이 `git status`, `git diff` 결과를 수집해 AI API를 호출하고, 변경 요약·커밋 메시지·PR 초안이 터미널에 출력된다. 사용자는 단일 실행으로 자동화 흐름이 끝까지 동작함을 확인할 수 있다.

2. GitHub 리포지토리

- 입력/요청: GitHub에 리포지토리를 생성하고, AI 기반 Git 커밋/PR 자동 생성기 프로젝트(소스 코드 폴더)를 원격 저장소에 push 한다.
- 출력/화면: GitHub 리포지토리에서 소스 코드/파일 구조가 정상적으로 업로드되어 있으며, 커밋 히스토리와 브랜치 작업 흐름을 확인할 수 있다.

3. 사용 가이드 문서(README.md)

- 입력/요청: 리포지토리의 README.md를 열어 실행 방법을 확인한다.
- 출력/화면: 설치 방법, 환경변수(API Key) 설정 방법, 실행 예시, 커밋/PR 생성 결과 예시, 주의사항(또는 운영 관점)이 포함되어 있어 사용자는 문서만 보고 도구를 실행할 수 있다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

1. AI API를 REST API 방식으로 연동하는 과정에서 요청을 구성하고, 응답을 처리하며, 예외를 대응하는 전체 흐름을 말로 설명할 수 있다.
2. `temperature`, `max_tokens` 등 주요 파라미터가 결과 품질에 어떤 영향을 주는지 구분하고 설명할 수 있다.
3. `git status`, `git diff` 등 Git 명령 실행 결과를 프로그램 입력으로 연결하는 방식과 자동화 흐름을 설명할 수 있다.
4. Commit/Pull Request 양식과 코드 변경 맥락을 포함해 AI가 요구사항에 맞는 요약을 생성하도록 프롬프트를 구성하는 원리를 설명할 수 있다.
5. 생성된 커밋/PR 텍스트가 실무 규칙(길이·템플릿·표현 제한)을 만족하도록 내용을 검증하고 필요한 부분을 다듬는 이유와 방법을 설명할 수 있다

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. Git 변경 사항 수집

- CLI 실행은 Git이 초기화된 프로젝트 루트 디렉토리에서 수행되어야 한다.
- CLI 실행 시 `git status` 결과를 수집하여 변경된 파일 목록을 확인할 수 있어야 한다.
- CLI 실행 시 `git diff` 결과를 수집하여 변경 내용(diff 텍스트)을 얻을 수 있어야 한다.
- 변경 사항이 없을 경우, “변경 사항이 없습니다”에 준하는 메시지를 출력하고 종료해야 한다.

2. AI API 연동

- AI API Key는 환경변수로 설정되어야 하며, 코드에 하드코딩하지 않는다.
- CLI를 실행하면 AI API가 호출되고, 생성된 커밋 메시지 또는 PR 초안 텍스트가 터미널에 출력되어야 한다.
- API 호출 실패(네트워크 오류, 인증 실패 등) 시 오류 원인을 포함한 메시지를 출력해야 한다.
- API 호출 파라미터(예: 모델, temperature, max_tokens)는 **CLI 옵션으로 변경 가능**해야 한다.
    - 예: `-model`, `-temperature`, `-max-tokens` 같은 옵션을 제공하고 기본값을 둔다.

3. 커밋 메시지 자동 생성

- `commit` 명령을 실행하면 커밋 메시지가 생성되어 터미널에 출력되어야 한다.
- 커밋 메시지는 변경 사항 요약을 기반으로 생성되어야 한다.
- 출력 결과에는 **커밋 제목 1줄이 필수로 포함**되어야 한다.
- 커밋 본문은 **선택적으로 포함**될 수 있다(본문 포함 여부는 구현 선택).
- (요약 품질 최소 기준) 커밋 본문을 포함하는 경우, 본문에 아래 중 1개 이상을 포함해야 한다.
    - 변경된 파일(또는 모듈) 1~3개 언급
    - 핵심 변경 사항 1~2개를 불릿으로 요약
- 생성된 커밋 메시지는 터미널에 출력되어 사용자가 복사하여 적용할 수 있어야 한다.

4. Pull Request 제목/본문 자동 생성

- CLI 실행 시, PR 제목/본문 초안을 생성하는 명령어를 실행할 수 있어야 한다.
- PR 본문은 템플릿 구조를 가져야 하며 아래 섹션 헤더를 포함해야 한다.

    - Why(변경 배경)
    - What(핵심 변경 사항)
    - How to Test(테스트 방법)
- 각 섹션에는 **최소 1개 이상의 불릿**이 포함되어야 한다.
- PR 제목은 1줄로 출력되어야 하며, PR 본문과 함께 터미널에서 확인할 수 있어야 한다.

5. 출력 형식 검증 및 다듬기

- 생성된 커밋/PR 텍스트는 아래 **길이/형식 규칙**을 적용할 수 있어야 한다(검증 후 재생성 또는 후처리 중 택1).
    - 커밋 제목: 50자 이내 권장(최대 72자)
    - PR 제목: 최대 80자
    - PR 본문: Why/What/How to Test 섹션 헤더 필수 + 각 섹션 최소 1개 불릿
- 사용자가 결과를 검토할 수 있도록, 최종 출력은 구분선/헤더 등으로 구획이 나뉘어 표시되어야 한다.

6. 리포지토리 및 문서화

- 결과물은 GitHub 리포지토리에 push 되어 있어야 한다.
- README.md에는 아래 내용이 포함되어야 한다.
    - 설치 및 실행 방법
    - 환경변수(API Key) 설정 방법
    - 커밋/PR 생성 명령 사용 예시
    - 출력 예시(커밋 메시지 / PR 본문 예시)
    - 아래 항목 중 1개 이상 포함
        - 민감정보 포함 가능성과 대응(마스킹/제외/안전 모드 안내)
        - 비용/요청 횟수 제한 및 권장 사용법

5. 보너스 과제 (선택)

1. 실제 리포지토리에 적용하여 PR 1건 완성하기

- 이전에 수행했던 미션 GitHub 리포지토리 1개를 선정한다.
- 기능/리팩토링/문서 개선 등 “의미 있는 변경”을 브랜치에서 수행한다.
- 본 도구로 커밋 메시지 1회 + PR 초안 1회를 생성해 실제 PR을 작성한다.

- 제출(증빙): PR 링크 1개 + “AI 초안 → 최종 PR” 변경점 요약본 (무엇을 왜 고쳤는지 5~10줄)

2. 커밋/PR 템플릿 커스터마이징

- 선택한 이전 미션 리포지토리의 기존 커밋/PR 스타일을 분석해 “팀 컨벤션”을 정의한다. (예: 커밋 prefix 규칙(feat/fix/docs), 스코프 표기, PR 템플릿(섹션/톤/체크리스트))
- 도구에 컨벤션을 반영하는 방법을 1개 이상 구현한다.

  (예: `.ai-gitgen.yml` 같은 설정 파일 / `--convention` 옵션 / 프롬프트 템플릿 교체 등)
- 제출(증빙): 컨벤션 문서(README 섹션) + 컨벤션 적용 전/후 생성 결과 비교 1회

3. 안전 모드 고도화

- `-safe-mode` 에서 마스킹 규칙을 확장하거나(정규표현식 기반), diff 전송 제한 정책(파일/줄 기준)을 조정 가능하게 만든다.
- 제출(증빙): safe-mode 정책(숫자 기준 포함) + safe-mode ON/OFF 결과 차이 비교

개발환경

6. 개발 환경

- Python 3.10 이상
- 웹 화면 없이, 터미널에서 실행되는 프로그램으로 구현한다.

제약조건

7. 제약 사항

- **AI API 사용 범위**
  - AI API Key는 환경변수로만 관리하며 코드에 하드코딩하지 않는다.
  - 과도한 비용 발생을 방지하기 위해, 1회 실행 시 요청 횟수는 1~2회 이내로 제한하는 것을 권장한다.

- (권장 구현) `commit / pr` 명령은 각각 AI API 1회 호출로 결과를 생성하고, 로그에 호출 횟수를 출력한다.

- **Git 연동 범위**
  - Git 변경 사항 수집은 `git status`, `git diff` 범위로 제한한다.
  - 커밋/PR 생성은 초안 텍스트 출력까지를 목표로 한다.
  - `git push`, GitHub PR 생성(API 연동) 등 원격 저장소 자동 반영 기능은 구현하지 않는다.

- **보안/민감정보**
  - 보안/민감정보: `git diff` 에 포함될 수 있는 민감정보(API Key, 개인정보 등)는 프롬프트에 포함하지 않도록 주의한다.
  - 아래 중 1개 이상을 `-safe-mode` 같은 옵션으로 제공한다.
    - (A) diff에서 특정 패턴(API Key 형태, 이메일 등)을 마스킹 후 전송
    - (B) diff 일부만 전송(예: 최대 10개 파일, 최대 200줄)
  - 생성된 커밋/PR 문구는 최종 정답이 아니며, 사용자가 검토 후 적용한다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구는 얼마든지 달라도 된다.

- **커밋 메시지 자동 생성 예시**
  - 실행(예시)

```
CSS
python [main.py](<http://main.py/>) commit
```

  - 터미널 출력(예시)

```
CSS
[INFO] Git status 수집 완료: 3개 파일 변경 감지
[INFO] Git diff 수집 완료: 128줄
[INFO] AI API 요청 중...
```

```
[DONE] 커밋 메시지 생성 완료

- -- Commit Message ---
feat: Git 변경 사항 기반 커밋 메시지 자동 생성 기능 추가

- git diff 결과를 수집해 AI 입력 컨텍스트로 전달하도록 구현
- 커밋 메시지 템플릿(feat/fix 등) 생성 규칙 적용
- API Key 미설정 시 안내 메시지 및 에러 처리 개선
----------------------
```

- **PR 제목/본문 자동 생성 예시**
  - 실행(예시)

```
CSS
python [main.py](<http://main.py/>) pr
```

  - 터미널 출력(예시)

```
CSS
[INFO] 현재 브랜치: feature/commit-pr-generator
[INFO] AI API 요청 중...
[DONE] PR 초안 생성 완료

- -- PR Title ---
feat: 커밋/PR 자동 생성 기능 추가

- -- PR Body ---
## Why
- 팀 협업 시 커밋 메시지와 PR 설명 작성에 시간이 소요되어 자동 생성 도구가 필요했습니다.
- Git 변경 사항을 기반으로 일관된 형식의 요약 텍스트를 생성해 리뷰 효율을 높이고자 했습니다.

## What
- git status, git diff 결과를 수집해 AI 입력 컨텍스트로 전달하는 로직 추가
- 커밋 메시지 자동 생성 기능(python [main.py](<http://main.py/>) commit) 구현
- PR 제목/본문 자동 생성 기능(python [main.py](<http://main.py/>) pr) 및
```

```
템플릿 적용
- API Key 누락/요청 실패 시 에러 메시지 및 예외 처리 개선

## How to Test
- 환경변수 설정: export AI_API_KEY="YOUR_KEY"
- 커밋 메시지 생성: python [main.py](<http://main.py/>) commit
- PR 초안 생성: python [main.py](<http://main.py/>) pr
- 결과로 출력된 PR 본문이 Why/What/How to Test 구조와 길이 규칙을 만족하는지 확인
```

- **AI API Key 미설정/오류 상황 예시**
  - 실행(예시)

```
python [main.py](<http://main.py/>) commit
```

  - 터미널 출력(예시)

```
[ERROR] AI_API_KEY 환경변수가 설정되지 않았습니다.
## 예) export AI_API_KEY="YOUR_KEY"
```

- **변경 사항이 없는 경우 예시**
  - 실행(예시)

```
python [main.py](<http://main.py/>) commit
```

  - 터미널 출력(예시)

```
[INFO] 변경 사항이 없습니다. 커밋 메시지를 생성하지 않고 종료합니다.
```